

Documentation – Hybridization Expansion CT-QMC solver version 3.0b1

Emanuel Gull,^{1,2} Hartmut Hafermann,³ and Philipp Werner⁴

¹*Max Planck Institute for the Physics of Complex Systems, Dresden, Germany*

²*Physics Department, University of Michigan, Ann Arbor, MI*

³*École Polytechnique, CNRS, 91128 Palaiseau Cedex, France*

⁴*Department of Physics, University of Fribourg, 1700 Fribourg, Switzerland*

(Dated: October 3, 2026)

This is the user documentation for the hybridization expansion solver of the ALPS DMFT code. It documents installation, running, and integration of the hybridization code into the ALPS self-consistencies of the ALPS DMFT code and the solver’s Python interface. It is designed to be a useful documentation for users, primarily in the LDA+DMFT community.

I. INTRODUCTION

Welcome! This is the user documentation for the third version of the ALPS¹ hybridization expansion² CT-QMC³ code, written by Emanuel Gull, in collaboration with Hartmut Hafermann and Philipp Werner, based on an earlier version⁴ by Philipp Werner, Emanuel Gull, Brigitte Surer, and Matthias Troyer. This user documentation is designed to make the program useful for users, in particular users from the LDA+DMFT field, who want to replace their solvers (IPT, Hirsch Fye, other versions of CT-QMC) with the ALPS hybridization code. A basic understanding of impurity models, Monte Carlo simulations and, where needed, LDA+DMFT is assumed. Knowledge of the inner workings of CT-QMC is not required, and we strive to make the code accessible to users who are not Monte Carlo experts.

This documentation, as well as version 3 of the hybridization expansion code, is open source. Bug reports and bug fixes, problem reports, and suggestion for improvements are most welcome.

II. PREREQUISITES

A. Downloading

The hybridization solver is included in the compiled ALPS packages as a binary. Release versions and nightly builds are available from alps.comp-phys.org. You can find the binary version at `alps-prefix/bin/hybridization`.

B. Building

The hybridization expansion code uses several ALPS libraries¹ and requires ALPS to build. You will therefore need to obtain the source code for ALPS from alps.comp-phys.org, either by downloading a current nightly build or by requesting a subversion account and downloading the svn version of ALPS. The ALPS home page has [detailed instructions](#) on how this works. In case of problems

building ALPS, the [ALPS user mailing list](#) will help you out. If this does not work, please don’t hesitate to contact [Emanuel](#).

The ALPS DMFT code is dependent on two additional modules: [HDF5](#) and [MPI](#) are required. Once ALPS and the hybridization code are installed in `$ALPS_PREFIX/bin/hybridization`, try running it. You will see

```
hybridization: No TOML run file specified
```

The command `hybridization --schema` prints every supported key with its type, default and bounds; the same schema is installed as `$ALPS_PREFIX/share/alps/schemas/cthyb.toml`.

If you have an MPI environment you can also run the code using, *e.g.*,

```
$MPI_RUN_COMMAND -np 3 $ALPS_PREFIX/ \
bin/hybridization
```

and you’ll see the same output three times. `$MPI_RUN_COMMAND` is platform dependent, common options are `mpirun`, `mpiexec`, `openmpi`, and so on. Consult your MPI manual on what you need to use.

III. RUNNING THE HYBRIDIZATION CODE

The input of the solver consists of a TOML run file, the hybridization function $\Delta(\tau)$ and optionally, the retarded interaction function $K(\tau)$ and its first derivative $K'(\tau)$.

A. Using the standalone executable

1. Specifying a run file

We start with a simple run file to solve a single site impurity problem. A minimal run file, call it `hyb1.toml`, could look like this:

```
[parameters]
SWEEPS = 100000000
THERMALIZATION = 1000
N_MEAS = 50
```

```

N_HISTOGRAM_ORDERS = 50
N_ORBITALS = 2
U = 4.0
MU = 2.0
N_TAU = 1000
BETA = 45.0

```

```

[input]
delta = "delta.dat"

[output]
results = "hyb1.out.h5"

```

```

[execution]
time_limit = 60
seed = 0

```

The [parameters] section describes a single impurity Anderson model (two “orbitals”, i.e. $N_{\text{orb}} = 2$, one for spin “up” and one for spin “down”) with $U = 4$, $\mu = 2$, and $\beta = 45$. The hybridization function is read from the file `delta.dat`, whose energy units are the units of the calculation. The [input], [output] and [execution] sections name the input files, the result archive and the run controls; this simulation runs for sixty seconds. Relative paths are resolved against the directory of the run file.

2. Running the solver

The `--validate` option checks the run file and its input data without sampling, as in `hybridization --validate hyb1.toml`. We can then run the program (make sure the directory of `hyb1.out.h5` is writable and the file `delta.dat` is present):

```
$ALPS_PREFIX/bin/hybridization hyb1.toml
```

A typical output is:

```

U matrix with 2 orbitals:
0 4
4 0
chemical potential with 2 orbitals:
2 2

```

```

local configuration:
0 empty
1 empty

```

the hybridization function is:

```

0 -0.5 -0.5
1 -0.492823 -0.492823
2 -0.485805 -0.485805
3 -0.478945 -0.478945
4 -0.47224 -0.47224
5 -0.465686 -0.465686
6 -0.459282 -0.459282

```

```

7 -0.453024 -0.453024
8 -0.446909 -0.446909
9 -0.440935 -0.440935
... *** etc *** ...
1000 -0.5 -0.5

```

```
process 0 starting simulation
```

Running the same simulation under MPI will produce additional lines indicating the process number: `process 1 starting simulation` and so on. After `execution.time_limit` is up or the number of SWEEPS have been done, the code exits. The results will be written to the file `hyb1.out.h5` named by `output.results`. The results can also be written in human-readable format by adding the line

```
text = true
```

to the [output] section; `output.text_directory` selects an existing directory for these files.

B. Python interface

The standalone `pyalps` wheel includes the solver module when built with application bindings enabled (the default). It can directly be called from within a Python script. This provides a flexible framework which allows one to easily set up tasks ranging from calculations for multiple parameters to complex selfconsistency schemes.

Basic usage of the Python interface is illustrated by the following script, which repeats the previous example for the standalone executable:

```

import pyalps.cthyb as cthyb # solver module
import pyalps.mpi as mpi     # MPI library

```

```

parms={
'SWEEPS'           : 100000000,
'THERMALIZATION'   : 1000,
'N_MEAS'           : 50,
'N_HISTOGRAM_ORDERS' : 50,
'N_ORBITALS'       : 2,
'U'                : 4.0,
'MU'               : 2.0,
'N_TAU'            : 1000,
'BETA'             : 45.0
}

```

```

if mpi.rank==0:
    f=open("delta.dat","w")
    for i in range(parms["N_TAU"]+1):
        f.write("%i %f %f\n"%(i,-0.5,-0.5))
    f.close()

```

```

run=cthyb.prepare(parms,
    input={'delta':'delta.dat'},
    output={'results':'results.out.h5',
            'text':True},

```

```
execution={'time_limit':60,'seed':0})
cthyb.solve(run)
```

The first line imports the solver module. The MPI module in the second line is needed for parallel runs; a serial run needs only the solver module. In the next four lines a simple (constant) hybridization function is written to file. The scientific parameters are specified in the form of a Python dict. The `prepare` method validates them together with the `input`, `output` and `execution` dictionaries, which correspond to the sections of a TOML run file, and returns the prepared run. Finally the solver is invoked by executing the `solve` method of the Python module on that run. The hybridization function is read from file, and results are written to the file `results.out.h5` named by `output.results`.

The script is executed as follows:

```
alpspython scriptname.py
```

or using MPI:

```
mpirun -np 2 alpspython scriptname.py
```

On some machines it may be necessary to use

```
mpirun -np 2 bash alpspython scriptname.py
```

instead. Note that on some platforms `mpirun` has to be replaced by a different command to invoke MPI. See your MPI manual for details.

The above example and more advanced examples on how to use the Python framework can be found in the hybridization tutorials within the `/tutorials` directory inside the ALPS installation directory:

```
/hybridization/01-python
/hybridization/02-kondo
/hybridization/03-retarded-interaction
/hybridization/04-spinfreezing
```

These tutorials illustrate how to run calculations for multiple parameters within a single script, plotting results, implementing a selfconsistency scheme, using the hdf5 interface, performing calculations with retarded interactions and for multiorbital models as well as performing simple computations with the measured observables.

IV. DETAILED INPUT DESCRIPTION

A. Hybridization function

We use the following definition of the hybridization function:

$$\Delta_{ab}(i\nu_n) = \sum_{kj} \frac{V_k^{aj} V_k^{*jb}}{i\nu_n - \epsilon_k^j} \quad (1)$$

$$\Delta(\tau) = \frac{1}{\beta} \sum_{n=-\infty}^{\infty} \Delta(i\nu_n) e^{-i\nu_n \tau} \quad (2)$$

where in the current implementation, the hybridization function is restricted to be diagonal, $\Delta_{ab}(\tau) = \Delta_a(\tau)\delta_{ab}$. As a consequence of this definition, the hybridization function is always negative. The code will throw a corresponding exception if this is not the case.

The hybridization function is read from a text file the name of which is specified by `input.delta`. The file has to contain a coordinate column and a column of hybridization values for each orbital. The coordinates are consecutive time indices starting at zero, or, with `input.delta_coordinate = "tau"`, the points of a uniform grid from 0 to β . The number of lines has to match the number of time points $N_\tau + 1$ used in the imaginary-time Green's function measurement, where N_τ is specified by the parameter `N_TAU`.

The hybridization function is read from an hdf5 archive if `input.delta_format = "hdf5"`. In this case the data has to be provided as a collection of one-dimensional arrays in the paths `/Delta_{i}`, one for each orbital i ($i = 0, \dots, N_{\text{orb}} - 1$). Each array has length $N_\tau + 1$ and the n -th entry corresponds to the value of the hybridization function at time $\tau_n = n\beta/N_\tau$ ($n = 0, \dots, N_\tau$). That the hybridization function has been read in correctly can be verified from the screen output of the solver immediately after starting the simulation.

Note that the level energies, or double counting terms should not be absorbed into the hybridization function. They have to be specified separately, see Sec. IV C.

In cases where the hybridization function becomes extremely small, of the order of the machine accuracy, numerical instabilities will occur, which render the results useless, since the algorithm operates on the *inverse* of the matrix of hybridization functions $[\Delta(\tau_i - \tau_j)]^{-1}$. This may happen within a DMFT calculation deep in the insulating phase and at very low temperature, or when an orbital is completely filled (or empty). These cases however are easily identified at the level of the Green's function, which will exhibit a lot of noise or even may change sign.

B. Retarded interaction function

A retarded interaction occurs in models with phonons, when dynamical screening is considered, or in the context of extended dynamical mean-field theory.

The general algorithm for phonons is given in Ref. 5. The formalism for retarded interactions is described in Ref. 6. The function $K(\tau)$, corresponding to the twice-integrated retarded interaction (for details see Ref. 6) connects all pairs of creation and annihilation operators. It is symmetric, $K(\beta - \tau) = K(-\tau) = K(\tau)$, and hence is tabulated as a function $K(\tau)$ on $N_\tau + 1$ points between 0 and β . K is positive and equal for all orbitals. We further use the convention that $K(0) = 0$. For the measurement of the improved estimator, the integrated retarded interaction, or equivalently, the first derivative of $K(\tau)$, i.e. $K'(\tau)$, is required. While in principle $K'(\tau)$

can be obtained from the knowledge of $K(\tau)$, the computation from finite differences on the imaginary time grid is not reliable. Hence the derivative should also be precomputed and be provided in tabulated form.

Providing the retarded interaction function and its derivative is optional. This feature is enabled by providing a filename through `input.retarded_interaction`. $K(\tau)$ and $K'(\tau)$ are provided either through a text file, or, if `input.retarded_interaction_format = "hdf5"`, within an hdf5 archive. The file format in both cases is similar as for the hybridization function. The text file should provide *three* columns. The first column is the coordinate, a time index or, with `input.retarded_interaction_coordinate = "tau"`, the time, the second column is the tabulated value of $K(\tau)$ and the third column gives the corresponding value of $K'(\tau)$. Likewise, the array in the hdf5 file should provide the $N_\tau + 1$ values of K and K' within the paths `/Ret_int_K` for $K(\tau)$ and `/Ret_int_Kp` for $K'(\tau)$. Note that there is no orbital dependence because the retarded interaction couples to all orbitals equally.

In case of a retarded interaction, the Hartree self-energy is given by

$$\Sigma_i^H = -2K'(0^+) \sum_i \langle n_i \rangle + \frac{1}{2} \sum_j U_{ij} \langle n_j \rangle, \quad (3)$$

so that in the particle-hole symmetric case an input chemical potential to $\mu = -2K'(0^+) + \sum_j U_{ij}/2$ (which is independent of i) will give a half-filled system. The chemical potential is provided through the parameters `U` and `MU` or the corresponding input files. The treatment of the retarded interaction requires the static interaction and chemical potential to be shifted by $U_{ij} \rightarrow U_{ij} - 2K'(0)$ for $i \neq j$ and $\mu \rightarrow \mu + K'(0)$, respectively. This is done internally by the solver and does not have to be done by the user. The shifted values will be printed to screen as “effective” interaction and chemical potential.

C. Orbital chemical potential / double counting terms

In the simplest case, the level energies of all levels are the same, and a parameter `MU` sets them to that value. Note that in this code, there is no shift of the chemical potential μ to half filling: For a single impurity Anderson model with interaction U , half filling is at $\mu = U/2$.

Double counting terms or crystal field splittings enter the code as orbital-dependent chemical potentials. They can be read in from a file, which is specified as `input.chemical_potential`. In the file the level energies should be listed consecutively, in the order of the orbitals. For example, a two-orbital case with a small magnetic field $H = 0.02$, half filled, for an interaction $U = 4$, could be specified like this:

```
1.99 2.01
```

After reading the on-site level energy it is printed to the standard output, as:

```
chemical potential with 2 orbitals:
1.99 2.01
```

The chemical potential can also be provided in the form of an hdf5 archive. To this end, set `input.chemical_potential_format = "hdf5"`. The level energies should be stored in a single array in the file path `/MUvector`, in the order of the orbitals.

D. General density-density interaction matrices

In the simplest case, repulsion terms are U . This is the default case that is chosen if only the parameter `U` is specified.

In multiorbital problems, more complicated interactions are needed. In this case, the “orbital index” i , which runs from 0 to $N_{\text{orb}} - 1$ used in the code represents a combined spin-orbital index $i = \{\sigma, m\}$. For example, for a model with two physical orbitals and two spins, we have $N_{\text{orb}} = 4$. The code in general makes no assumption on the order of spins and physical orbitals. We recommend ordering these as follows: $1 \uparrow, 1 \downarrow, 2 \uparrow, 2 \downarrow$, which correspond to orbital indices $i = 0, 1, 2, 3$, respectively. The values of the interaction matrix can be specified in a file `input.interaction_matrix`, which has $N_{\text{orb}} \times N_{\text{orb}}$ entries specifying the matrix elements U_{ij} of the interaction term $\frac{1}{2} \sum_{ij} U_{ij} n_i n_j$.

To see how the interaction matrix for the above ordering is built consider the following Hamiltonian:

$$\begin{aligned} H_U = & \frac{1}{2} U \sum_{m,\sigma} n_{m\sigma} n_{m,-\sigma} \\ & + \frac{1}{2} U' \sum_{m \neq m', \sigma} n_{m,\sigma} n_{m',-\sigma} \\ & + \frac{1}{2} (U' - J) \sum_{m \neq m', \sigma} n_{m,\sigma} n_{m',\sigma}, \end{aligned} \quad (4)$$

$$\hat{U} = \begin{pmatrix} 0 & U & U'-J & U' \\ U & 0 & U' & U'-J \\ U'-J & U' & 0 & U \\ U' & U'-J & U & 0 \end{pmatrix} \quad (5)$$

Choosing $U = 4.0$, $U' = 3.6$ and $J = 0.2$, the file generating the interaction matrix of the example above is:

```
0.0 4.0 3.4 3.6
4.0 0.0 3.6 3.4
3.4 3.6 0.0 4.0
3.6 3.4 4.0 0.0
```

After reading the interactions, they are printed to the standard output like this:

```

U matrix with 4 orbitals:
0 4 3.4 3.6
4 0 3.6 3.4
3.4 3.6 0 4
3.6 3.4 4 0
chemical potential with 4 orbitals:
5.5 5.5 5.5 5.5

```

Since the above Hamiltonian is widely used, it is directly into the code. To use it, simply specify the parameters U, U' and J in the run file: adding the lines

```

U = 4.0
"U'" = 3.6
J = 0.2

```

to the `[parameters]` section produces the same output as above. For a single physical orbital, it reduces to the Hubbard interaction $Un_{\uparrow}n_{\downarrow}$. Note that when using the built-in Hamiltonian, the order of spins and orbitals *is* relevant and has to be chosen as recommended above. For these interaction matrices, the condition for half-filling reads $\mu_{1/2} = \frac{1}{2} \sum_i \hat{U}_{ij}$ (which is independent of j). For the above example, $\mu_{1/2} = 5.5$. The interaction matrix will be read from an hdf5 archive if `input.interaction_format = "hdf5"`. The N_{orb}^2 values should be stored in the file path `/Umatrix` in a one-dimensional array, where U_{ij} is stored at position $iN_{\text{orb}} + j$, i.e. elements within a row are contiguous.

V. DETAILED PARAMETER DESCRIPTION

A run file has four sections. `[parameters]` holds the scientific parameters described below, `[input]` names the input files, `[output]` names the result files and `[execution]` controls the run. The command `hybridization --schema` lists every key with its type, default and bounds. Boolean parameters take the TOML values `true` and `false`.

A. Mandatory parameters

The following parameters are required for any simulation. The code will not work unless all of them are provided.

- **N_MEAS**= $\{\text{natural number}\}$ specifies how many updates need to be done between measurements. For e.g. **N_MEAS**=50, 50 modifications of the Monte Carlo configuration (such as segment insertions and removals) are attempted between measurements.
- **SWEEPS**= $\{\text{natural number}\}$ is the total number of Monte Carlo sweeps. A sweep consists of **N_MEAS** attempted modifications of the Monte Carlo configuration and a single the measurement of the observables. The code exits either after it has done at least **SWEEPS** Monte Carlo sweeps or after the maximum time has been reached.

- **THERMALIZATION**= $\{\text{natural number}\}$ is the number of thermalization steps. These are steps that are done at the beginning that are needed to reach the equilibrium distribution.
- **N_ORBITALS**= $\{\text{natural number}\}$ is the number of (spin-)orbitals. For the single impurity Anderson model this is two (spin up and spin down), Cerium would have 14.
- **N_TAU**= $\{\text{natural number}\}$ specifies the number N_{τ} of τ -points on which the Green function is measured, and also indicates the number of τ -points on which the hybridization function and the retarded interaction are specified. The imaginary time at index n is given by $\tau_n = n\beta/N_{\tau}$, where $n = 0, \dots, N_{\tau}$, so that the first point (index $n = 0$) specifies $G(\tau = 0)$, the last point (index $n = N_{\tau}$) specifies $G(\tau = \beta)$. Note that the number of imaginary time points hence is $N_{\tau} + 1$.
- **BETA**= $\{\text{real number}\}$ specifies the inverse temperature $\beta = T^{-1}$.
- **U**= $\{\text{real number}\}$ is the on-site interaction. In this case it is a term $\sum_i n_{i\uparrow}n_{i\downarrow}$. It is not needed when an interaction matrix is read from `input.interaction_matrix`, see Sec. [IV D](#).
- **input.delta** = *filename* specifies the name of the file which contains the hybridization function input data.
- **output.results** = *filename* specifies the name of the hdf5 file to which the results are written.

B. Optional parameters

1. Physical parameters

The interaction parameters are explained in Sec. [IV D](#).

- **MU**= $\{\text{real number}\}$ is the chemical potential or level energy for model Hamiltonians; the default is 0. On-site level energies can be specified differently (in particular in an orbital dependent way, see section on double counting). This parameter is ignored if a file with the chemical potential values is specified using `input.chemical_potential`.
- **N_HISTOGRAM_ORDERS**= $\{\text{natural number}\}$ is the maximum expansion order to which the order histogram is measured; the default is 50.
- **J**= $\{\text{real number}\}$ specifies the interaction parameters J .
- **U'**= $\{\text{real number}\}$ specifies the interaction parameters U' .

2. Measurement control parameters

The following optional parameters control the different measurements implemented in the solver. For details on those measurements, see Sec. VI.

- **MEASURE_time**={false,true} activates the Green's function measurement in imaginary time. Note: This measurement is turned on by default.
- **MEASURE_freq**={false,true} activates the Green's function measurement on Matsubara frequencies. Requires **N_MATSUBARA**.
- **N_MATSUBARA**={*natural number*} The number N_ν of (fermionic) Matsubara frequencies $\nu_n = (2n + 1)\pi/\beta$. The Green's function will be measured for all frequencies with $n = 0, \dots, N_\nu - 1$.
- **MEASURE_legendre**={false,true} activates the measurement of coefficients of Green's function in the Legendre polynomial basis. Requires **N_LEGENDRE** and **N_MATSUBARA**.
- **N_LEGENDRE**={*natural number*} specifies the number of Legendre coefficients to be measured. Coefficients with indices $l = 0, \dots, N_l - 1$ will be measured.
- **MEASURE_nn**={false,true} controls the measurement of the equal-time density-density correlation function.
- **MEASURE_nnt**={false,true} turns on or off the measurement of the time dependent density-density correlation function. Requires **N_nn**.
- **N_nn**={*natural number*} specifies the number of imaginary time points for which the density-density correlation function is measured.
- **MEASURE_g2w**={false,true} turns on the measurement of the two-particle Green's function. Requires **N_w2** and **N_W**.
- **MEASURE_h2w**={false,true} turns on the measurement of the three-particle correlator H (see Sec.??). Requires **N_w2** and **N_W**.
- **N_w2**={*natural number*} specifies the number $\tilde{N}_\nu$ of fermionic frequencies for the two-particle Green's function or the correlator H . These quantities are measured for frequencies with indices $n = -\tilde{N}_\nu/2, \dots, \tilde{N}_\nu/2 - 1$.
- **N_W**={*natural number*} specifies the number N_ω of bosonic frequencies $\omega_m = 2\pi m/\beta$ for the two-particle Green's function or the correlator H . These quantities are measured for frequencies with indices $m = 0, \dots, N_\omega - 1$.

- **MEASURE_nnw**={false,true} turns on the measurement of the susceptibility in frequency. Requires **N_W**.
- **MEASURE_sector_statistics**={false,true} controls the measurement of the sector statistics.

3. Optional control parameters

- **SPINFLIP**={false,true} specifies whether to perform spin flip updates. In such an update a segment is moved from one orbital to another, if the latter is empty.
- **COMPUTE_VERTEX**={false,true} specifies whether the reducible impurity vertex function should be calculated. This parameter requires **MEASURE_g2w** or **MEASURE_h2w** or both be set to true.
- **VERBOSE**={false,true} When turned on, some additional information (e.g. which measurements are turned on) is displayed when starting the simulation.

4. Input files

- **input.delta_format** = "text" (default) or "hdf5" specifies whether the hybridization function is read from an hdf5 archive. The data must be stored in the paths /Delta_0, /Delta_1, etc. For details on the file format, see Sec. IV A.
- **input.delta_coordinate** = "index" (default) or "tau" specifies whether the first column of a text hybridization file holds time indices or imaginary times.
- **input.chemical_potential** = *filename* specifies the file name for a text file containing the orbital dependent chemical potentials. The file format is such that the chemical potentials for all orbitals are listed consecutively, separated either by a space or a new line. If this file is specified, the parameter **MU** is ignored.
- **input.chemical_potential_format** = "text" (default) or "hdf5" specifies whether the chemical potential should be read from an hdf5 archive. The data must be stored in the path /MUvector as an array of length N_{orb} . See Sec. IV C for more information.
- **input.interaction_matrix** = *filename* specifies the file name for a text file containing the interaction matrix. For the file format, refer to Sec. IV D. If this file is specified, the parameter **U** is ignored.

- `input.interaction_format = "text"` (default) or `"hdf5"` specifies whether the interaction matrix should be read from an hdf5 archive. The data must be stored in the path `/Umatrix`. For the file format, see Sec. IV D.
- `input.retarded_interaction = filename` specifies the filename for the retarded interaction function K . When specified, this feature is automatically turned on.
- `input.retarded_interaction_format = "text"` (default) or `"hdf5"` specifies whether the retarded interaction is read from an hdf5 archive. The data must be stored in the paths `/Ret_int_K` and `/Ret_int_Kp`. For the format, consult Sec. IV B.
- `input.retarded_interaction_coordinate = "index"` (default) or `"tau"` specifies whether the first column of a text retarded-interaction file holds time indices or imaginary times.

5. Output and execution

- `output.text = false` (default) or `true` specifies if the simulation results should be written to text-files in human-readable format (in addition to the hdf5 output).
- `output.text_directory = directory` selects an existing directory for the text files; the default is the directory of the run file.
- `output.base_path = hdf5 path` selects the hdf5 group under which the scientific results are written.
- `execution.time_limit = natural number` is the maximum runtime of the code, in seconds. The default 0 disables the limit, and the run ends after SWEEPS sweeps.
- `execution.seed = natural number` is the random number seed.
- `execution.progress_period = natural number` If `VERBOSE = true`, it specifies the number of sweeps after which simulation details of the master thread (acceptance ratios etc.) are printed to `stdout`. The default value is 100 000.

VI. DETAILED DESCRIPTION OF MEASUREMENTS

In this section, we describe the measurements supported by the code in detail. That is, we state what is measured exactly, and give the precise definition of the observable that we use, so that the code can be easily

integrated into your projects. We further give some recommendations on how to use these measurements.

All measurements except for the one for the imaginary-time Green's function are performed once after `N_MEAS` attempted configuration changes.

A. Imaginary-time Green's function

We define the imaginary time Green's function as follows:

$$G_i(\tau) := -\langle T_\tau c_i(\tau) c_i^\dagger(0^+) \rangle \quad (6)$$

The code measures this function by binning it on an equidistant grid on the interval $[0, \beta]$ with the number of bins N_τ specified by the parameter `N_TAU`. The corresponding improved estimator

$$F_i(\tau) := -\frac{1}{2} \sum_j (U_{ij} + U_{ji}) \langle T_\tau n_j(\tau) c_i(\tau) c_i^\dagger(0^+) \rangle \quad (7)$$

is measured automatically. The bins of width β/N_τ are centered around the equidistant grid points at $\tau_n = n\beta/N_\tau$. Hence $\tau_0 = 0$ and $\tau_{N_\tau} = \beta$. Note that because of this choice, the first and the last bin only have half the width of a regular bin. As a consequence of the above definition, the density is given by $-G(\beta - 0^+) = G_i(\tau = 0^-) = -\langle T_\tau c_i(0^-) c_i^\dagger(0^+) \rangle = +\langle c_i^\dagger(0^+) c_i(0^-) \rangle = \langle c_i^\dagger c_i \rangle = \langle n_i \rangle$.

This measurement is very efficient and the performance of the algorithm is hardly influenced by number of bins, which hence can be chosen large. We recommend to use a number of bins of the order of at least $\sim 5\beta U$. The measurement is turned on by default, but can nevertheless be turned off by setting `MEASURE_time = false`.

The raw data of the measurement is stored in the hdf5 output file in the path `/simulation/results/g_i/mean/value`, where i is the orbital index. Note that the data at the interval endpoints has less statistics because of the smaller bin size. When using the raw data, the values at the interval endpoints have to be multiplied by two (since the effective bin size at the boundary is half of the regular size) to get the correct value. The data for all orbitals is stored in a single one-dimensional array, where successive τ -points for a given orbital are consecutive in memory. The Green's function at time τ_n , $n = 0, \dots, N_\tau$ and orbital $i = 0, \dots, N_{\text{orb}} - 1$ is hence stored at position $i(N_\tau + 1) + n$. During post processing, the data for $G(\tau)$ is written to the path `/G_tau/0/mean/value` for orbital 0 and likewise for the other orbitals in the output file. The values at the interval endpoints are thereby replaced by the corresponding values inferred from the densities. When `output.text = true`, the processed Green's function data is written to a human-readable text file `Gt.dat`, which allows for easy plotting, e.g. with gnuplot. The first column is the imaginary time and successive columns are listed in the order of the orbitals.

B. Frequency Space Measurements

The code measures the Fourier transform of Green's function

$$G(i\nu_n) = \int_0^\beta d\tau G(\tau) e^{i\nu_n \tau}, \quad (8)$$

where $G(\tau)$ is defined as in (6).

Measurements in frequency space are relatively expensive, as $\exp(i\nu_n(\tau_i - \tau_j))$ has to be evaluated at each measurement for each pair of operators (values for successive frequencies are generated by multiplication of these exponentials). Frequency space measurements are enabled by setting `MEASURE_freq = true` and specifying `N_MATSUBARA=512` for $N_\nu = 512$ Matsubara frequencies. The Green's function is measured for frequencies ν_n with indices $n = 0, \dots, N_\nu - 1$. The number of frequencies compatible with our above recommendation for the number of time slices is $N_\nu \sim 5\beta U/2\pi$.

The improved estimators $F_i(i\nu_n)$ described in Ref. 7 only cause a small overhead and are automatically measured (using the same definitions as given in that reference). The resulting $F(i\nu_n)$ and $G(i\nu_n)$, as well as the self-energy $\Sigma(i\nu_n)$, are stored in the hdf5 file under `/G_omega`, `/F_omega`, and `/S_omega`, respectively, where the actual data is in the subpath `/i/mean/value` for orbital $i \in 0, \dots, N_{\text{orb}} - 1$. The data for each of these quantities is stored in *two*-dimensional array with dimensions $N_\nu N_{\text{orb}} \times 2$. The index of the first dimension specifies the location $iN_\nu + n$ of the complex value for frequency ν_n and orbital i , and the second index ($\{0, 1\}$) selects real or imaginary part. The raw data for $G(i\nu_n)$ and $F(i\nu_n)$ is stored in path `simulation/results/<g/f>w_<re/im>_i`, where, e.g., the subpath is `gw_re_0` for the real part of G in orbital 0.

If text output is enabled (`output.text = true`), G , F , and Σ are also written to the text files `Gw.dat`, `Fw.dat`, and `Sw.dat`, respectively, where the first column is the Matsubara frequency and the i -th pair of columns lists real and imaginary part of the i -th orbital.

C. Legendre Polynomial Measurements

The measurements can also be done using Legendre polynomials – a method recently introduced by Boehnke *et al.*⁸. The code measures the coefficients of the expansion of Green's function in terms of orthogonal Legendre polynomials. These coefficients are defined as

$$G_l = \int_0^\beta d\tau G(\tau) P_l(x(\tau)) \quad (9)$$

where $x(\tau) = 2\tau/\beta - 1$. Note that this definition for the Legendre coefficients differs from the one in Ref. 8 by a factor $\sqrt{2l+1}$, which we omit to save time during the simulation and reintroduce during post processing. Legendre measurements are enabled by specifying

`MEASURE_legendre = true` and the number of Legendre coefficients N_l through `N_LEGENDRE`. The improved estimators F^7 are measured automatically.

The raw data (i.e. the coefficients) for orbital i are stored in the hdf5 output file in the path `/simulation/results/gl_i` and `/simulation/results/fl_i`. The actual values are stored in a one-dimensional array in the subpath `/mean/value`. After the simulation, the quantities $G(\tau)$, $F(\tau)$ as well as $G(i\nu_n)$, $F(i\nu_n)$ and $\Sigma(i\nu_n)$ are evaluated from the Legendre coefficients and stored in the paths `G_l_tau`, `F_l_tau`, `G_l_omega`, `F_l_omega` and `S_l_omega`, respectively. The values actual in orbital i are found within the one-dimensional array stored in the subpath `/i/mean/value`. For the evaluation, the number of imaginary time points and Matsubara frequencies is determined from the parameters `N_TAU` and `N_MATSUBARA`. If text output is enabled (`output.text = true`), G , F , and Σ are also written to the text files `Gt1.dat`, `Ft1.dat`, `Gw1.dat`, `Fw1.dat`, and `Sw1.dat`.

This measurement yields comparatively smooth curves, but as the results for different frequencies are correlated a precise error estimate and accurate control of the number of legendre coefficients considered is important.

The number of Legendre coefficients required for the accurate representation of a given observable may depend on the quantity under consideration and is difficult to infer from looking at the coefficients themselves. Rather one should analyze the dependence of a given observable on the basis cutoff N_l . To this end, one can look at the files `G1_conv.dat` and `F1_conv.dat` which are written when text output is enabled. In the former, the Green's function at $\tau = 0$ and $\tau = \beta/2$ is written as a function of the basis cutoff up to the maximum cutoff N_l . The latter contains the same for the correlator F . To get a feeling for the required cutoff, one should run a simulation at the desired parameters with a large cutoff N_l and plot the data in these files. For a converged calculation and sufficiently large N_l , one should see an extended plateau, where the observable does not change as a function of the cutoff. For less coefficients, the expansion is not converged and for a larger number, Monte Carlo noise enters. The basis cutoff is well defined for an N_l within the plateau.

Recently, it has been proposed to improve this method by employing the Kernel polynomial method⁹. We do not provide an implementation here. While the method does damp spurious oscillations in the Green's function, it leads to rather large systematic errors in the moments.

D. Density

The densities are always measured. They are stored in the hdf5 file in path `/simulation/results/density_i`. If text output is enabled, they are written to the file

observables.dat.

E. Susceptibility – imaginary time

The time-dependent density-density correlation function, $\chi_{ij}(\tau) = \langle n_i(\tau)n_j(0) \rangle$, or susceptibility, is measured by setting `MEASURE_nnt = true`. Only components with $j \leq i$ are measured. The spin and charge susceptibilities can be obtained as linear combinations of these correlation functions. The number of imaginary time points can be different from the one for Green's function and has to be specified through the mandatory parameter `N_nn`. The imaginary-time grid is otherwise the same as for Green's function. Note however, that because of the way the algorithm works, these functions are not binned, but measured exactly at the grid points. Hence the raw data does not have to be rescaled at the interval endpoints (cf. Sec. VIA). The raw data is stored in the hdf5 output file in the path `simulation/results/nnt_i_j`, within the subpath `/mean/value`. It is additionally written to the path `/nnt_i_j` without modification. When text output is enabled, the data is written to the text file `nnt.dat` for all $j \leq i$. The imaginary time is listed in column 1 and the susceptibility $\chi_{ij}(\tau)$ resides in column $1 + j + \frac{1}{2}i(i+1)$, i.e. χ_{00} column 1, χ_{10} in column 2 and χ_{11} in column 3, etc. Alternatively to the imaginary time measurement, the susceptibility can be measured directly in frequency (see the following section).

F. Susceptibility – frequency

In the frequency measurement for the susceptibility, the algorithm computes

$$\chi_{ij}(i\omega_m) = \int_0^\beta d\tau \chi_{ij}(\tau) e^{i\omega_m \tau} \quad (10)$$

where $\omega_m = 2m\pi/\beta$. Note that this measurement does not involve any time discretization. It is particularly useful and fastest if only the static susceptibility $\chi_{ij}(\omega = 0)$ is desired. In most cases, i.e. if the perturbation order is not exceptionally large, it is still faster for a finite number of frequencies compared to the imaginary-time measurement (using N_τ time points and an "equivalent" number of frequency points $N_\omega \sim N_\tau/2\pi$, respectively). Only the real part is measured, since the function is symmetric around $\beta/2$. The measurement is turned on with `MEASURE_nnw = true`. The number of bosonic frequencies is determined by the (mandatory) parameter `N_W`. Note that the same parameter determines the number of bosonic frequencies for measurements of two-particle correlation functions (cf Sec. VII). The raw data is stored in the hdf5 output file in the path `simulation/results/nnw_re_i_j`, within the subpath `/mean/value`. It is additionally written to the path `/nnw_re_i_j` without modification. When text output is

enabled, the data is written to the text file `nnw.dat` for all $j \leq i$. The frequency is listed in column 1 and the susceptibility $\chi_{ij}(\omega)$ resides in column $1 + j + \frac{i}{2}(i+1)$.

G. Equal-time density-density correlation functions

The measurement for the density-density (equal-time) correlation functions $\langle n_i n_j \rangle$ is turned on by letting `MEASURE_nn = true`. Although these are a special case of the time-dependent correlation functions above, we provide a separate measurement. They are measured for $j < i$ only, because the operators commute, $\langle n_i n_j \rangle = \langle n_j n_i \rangle$, and $\langle n_i n_i \rangle = \langle n_i^2 \rangle = \langle n_i \rangle$ would yield the density. The raw data is stored in the path `/simulation/results/nn_i_j`. No further post processing occurs. When text output is enabled, the values are appended to the file `observables.dat`.

H. Sector statistics measurement

The sector statistics give information on the fraction of (imaginary) time the impurity spends in a given state. The total number of possible states for this algorithm is $2^{N_{\text{orb}}}$. We can represent any of these states in the form

$$|n_0 = \{0,1\} n_1 = \{0,1\} \dots n_{N_{\text{orb}}-1} = \{0,1\}\rangle \quad (11)$$

To be specific, for a two-orbital impurity, there are four different states: The impurity can be unoccupied, which corresponds to the state $|00\rangle$. Orbital 0 (spin up, say) can be occupied while orbital 1 (spin down) is empty, $|10\rangle$, or orbital 1 can be occupied with 0 being empty $|01\rangle$. Finally, the impurity can be doubly occupied, i.e. in state $|11\rangle$. The result of a calculation for such a system at half-filling could look as follows:

```
#state |n_1={0,1} n_2={0,1} ...> n_i={0,1}: \
orbital i {empty,occupied}
#rel weight (in %)
0 18.9302 |00>
1 31.0485 |10>
2 31.0745 |01>
3 18.9467 |11>
```

To understand this, recall that in the atomic limit (no hybridization), the energy is given by $Un_\uparrow n_\downarrow - \mu(n_\uparrow + n_\downarrow)$. Hence, at half filling, where $\mu = U/2$, the energy of the singly occupied states is $-\mu$ and hence they are degenerate. The unoccupied and doubly occupied states are degenerate as well, with energy 0 and $U - 2\mu = 0$, respectively. Hence their contributions should be equal (within the Monte Carlo error). The singly occupied states are lower in energy and so their contribution is higher. Since the model is particle-hole symmetric, we further know that the time spent in states $|00\rangle$ and $|01\rangle$ must be the same as for $|10\rangle$ and $|11\rangle$, which can readily be seen from the data. The fraction the impurity

spends in state $|11\rangle$ corresponds to the integrated overlap and hence should be equal to the equal-time correlation function $\langle n_{\downarrow} n_{\uparrow} \rangle$, as can be verified from the file `observables.dat` (if `MEASURE_nn = true`):

```
n0=0.499892;
n1=0.500191;
n1n0=0.189423;
```

Note that the values are slightly different since they are measured in different ways. They will converge to one another as statistics is increased.

The data is stored in the hdf5 output file in the path `simulation/results/sector_statistics/mean/value` in a vector of length $2^{N_{\text{orb}}}$. The position of a given state in this vector is simply the decimal representation of the binary number encoding the state, i.e. the state $|01\rangle$ has the index $0 \cdot 2^0 + 1 \cdot 2^1 = 2$. With text output enabled, a file `sector_statistics.dat` as shown above is produced.

I. Two-particle correlation functions

The use of two-particle correlation functions is gaining importance. They are used for calculating lattice susceptibilities within dynamical mean-field theory or novel diagrammatic extensions of DMFT. To meet these requirements, we provide a measurement for the two-particle Green's function:

$$G_{ij}^{(2)}(\tau_a, \tau_b, \tau_c, \tau_d) := \langle c_i(\tau_a) c_i^\dagger(\tau_b) c_j(\tau_c) c_j^\dagger(\tau_d) \rangle \quad (12)$$

where $G_{ij}^{(2)}$ is a shorthand notation for $G_{ijji}^{(2)}$. As a result of the fact that the hybridization is diagonal, the Green's function depends only on two independent orbital indices. An annihilator on orbital i has to come in pair with a creator on the same orbital, otherwise the average is zero. To generate the orbital combinations $G_{ijji}^{(2)}$, relations of the form $G_{ijji}^{(2)}(\tau_a, \tau_b, \tau_c, \tau_d) = -G_{iijj}^{(2)}(\tau_a, \tau_d, \tau_c, \tau_b)$ can be used (and the corresponding ones in frequency space). These functions are measured only for $j \leq i$, since measuring this quantity with indices i and j interchanged would yield exactly the same result, i.e. there is no gain through averaging. The diagonal quantities ($i = j$) are different for different orbitals even if they are degenerate due to the sampling error, so that they can (and should) be averaged if the orbitals are symmetry equivalent.

Due to time translational invariance, the function depends on three independent time differences, or equivalently three independent frequencies. Because of memory

restrictions, the actual measurement is performed in frequency space, for which we use the following definition of the Fourier transform:

$$G_{ij}^{(2)}(\nu, \nu', \omega) := \frac{1}{\beta} \int_0^\beta d\tau_a \int_0^\beta d\tau_b \int_0^\beta d\tau_c \int_0^\beta d\tau_d \quad (13)$$

$$\times G_{ij}^{(2)}(\tau_a, \tau_b, \tau_c, \tau_d)$$

$$\times e^{i(\nu+\omega)\tau_a} e^{-i\nu\tau_b} e^{-i\nu'\tau_c} e^{-i(\nu'+\omega)\tau_d}$$

Here $\nu \equiv \nu_n = (2n+1)\pi/\beta$ and ν' are fermionic frequencies, whereas $\omega \equiv \omega_m = 2m\pi/\beta$ is bosonic. The number of fermionic frequencies $N_\nu^{(2)}$ is specified through the parameter `N_w2` and the number of bosonic ones (N_ω) through `N_W` (mind the capital "W"). The vertex is measured for fermionic frequencies with indices $n = -N_\nu/2, \dots, N_\nu/2 - 1$ and indices $m = 0, \dots, N_\omega - 1$ for the bosonic frequency. For negative bosonic frequencies, the relation $G_{ij}^{(2)}(\nu, \nu', -\omega) = G_{ij}^{(2)*}(-\nu, -\nu', \omega)$ holds.

The real and imaginary part of the two-particle Green's function are stored in the hdf5 output file in the path `simulation/results` as `g2w_re_i_j` and `g2w_im_i_j`, respectively ($j \leq i$). The actual data is stored in a one-dimensional array of dimension $N_\nu^{(2)} N_\nu^{(2)} N_\omega$ in the subpath `/mean/value`. The value corresponding to frequencies ν_n , $\nu_{n'}$ and ω_m is stored at position $m(N_\nu^{(2)})^2 + (n + N_\nu^{(2)}/2)N_\nu^{(2)} + (n' + N_\nu^{(2)}/2)$ (note that $n = -N_\nu^{(2)}/2, \dots, N_\nu^{(2)}/2 - 1$). When text output is enabled, the two-particle Green's function is written to the file `g2w.dat`. The format allows to easily plot the data as a function of a single frequency using `gnuplot`, for example:

```
gnuplot> p "<cat g2w.dat |grep 'wp: 1 '\ | \
grep 'W: 1 '\ | grep 'i: 0 j: 0' " u 2:11 w lp
```

(note the whitespace in `'W: 1 '`). Column 11 is the real part, column 12 the imaginary part.

Due to the possibly large number of observables (frequencies) that have to be measured, this measurement may significantly slow down the code, or even exceed memory available to each process. In such cases, it may be desirable to use additional symmetries, if present, or altered frequency meshes. In addition, different methods may have different requirements and for these reasons, it is impossible to provide a one-fits-all implementation. However, the advanced user may regard the function `void hybridization::measure_G2w` in the source file `hybmeasurements.cpp` as an API and modify it according to her needs.

The code may also be used to calculate the vertex function of the impurity model, which is defined as

$$\gamma_{ij}(\nu, \nu', \omega) = \frac{G_{ij}^{(2)}(\nu, \nu', \omega) - \beta[G_i(\nu + \omega)G_j(\nu')\delta_{\omega,0} - \delta_{ij}G_i(\nu + \omega)G(\nu')\delta_{\nu\nu'}]}{G_i(\nu + \omega)G_i(\nu)G_j(\nu')G_j(\nu' + \omega)}. \quad (14)$$

We provide the measurement of the correlation function

$$H_{ij}^a(\tau_1\tau_2\tau_3\tau_4) := \langle n_a(\tau_1)c_i(\tau_1)c_i^\dagger(\tau_2)c_j(\tau_3)c_j^\dagger(\tau_4) \rangle, \quad (15)$$

which represents an improved estimator for the vertex function⁷ and will be measured in frequency if `MEASURE_h2w` is set to one. The vertex function is computed during post processing, if `COMPUTE_VERTEX = true`. In addition, the frequency measurement (`MEASURE_freq`) and at least one of the measurements `MEASURE_g2w` or `MEASURE_h2w` (or both) have to be turned on. The vertex will be evaluated according to the data available. Determining the vertex from $G^{(2)}$ only is least accurate. Using $G^{(2)}$ and H is the most accurate. The vertex may also be calculated from H only. This is somewhat less accurate, but saves a factor of 2 in memory. The number of frequencies for the single-particle Green's function that enters Eq. (14), has to fulfill the relation $N_\nu \leq N_\nu^{(2)} + N_\omega - 1$, otherwise the code will not work. There are different ways to evaluate the vertex. Depending on which of the latter two measurements are turned on, the method that yields the most accurate results will be chosen. The function H is stored analogously to the two-particle Green's function, as `h2w_re_i_j` and `h2w_im_i_j`, respectively and written to file `h2w.dat` when text output is enabled. When evaluated, the vertex function is written to the hdf5 output file in the path `/vertex_i_j` in a *two-dimensional* array with dimensions $N_\nu^{(2)}N_\nu^{(2)}N_\omega \times 2$. The index of the first dimension specifies the position $m(N_\nu^{(2)})^2 + (n + N_\nu^{(2)}/2)N_\nu^{(2)} + (n' + N_\nu^{(2)}/2)$ of the complex value for frequencies ν_n , $\nu_{n'}$ and ω_m and the second index ($\{0,1\}$) selects real or imaginary part. With text output enabled, the vertex is written to the file `gammaw.dat`, in the same format as the two-particle Green's function.

Note that this vertex function is the full (i.e. reducible) vertex of the impurity model. If the irreducible or even fully irreducible vertex is desired, this has to be obtained by inverting a Bethe-Salpeter equation or from inverse Parquet, respectively.

VII. DMFT – SPECIFYING A SELF-CONSISTENCY

To run the impurity solver in a DMFT self-consistency, the output $G(\tau)$ has to be input into a self-consistency condition and a hybridization function $\Delta(\tau)$ has to be produced.^{10,11} In the simplest case, for a DMFT self-consistency for the Bethe lattice with infinite coordination number (resulting in a semicircular density of states), we obtain $\Delta(\tau) = t^2 G(\tau)$. Note that in our code, both Δ and $G(\tau)$ are negative for $\tau > 0$. In practice, more elaborate self-consistency conditions are almost always needed. Some of them are included in the ALPS DMFT framework, but in general they are user provided.

A. ALPS DMFT framework

The hybridization solver is used in several ALPS tutorials, which illustrate the use of the solver with the ALPS DMFT framework. This comprises the tutorials in the following directories:

```
05-dmft/02-hybridization
05-dmft/04-mott
05-dmft/05-osmt
05-dmft/06-paramagnet/hyb
05-dmft/08-lattices
```

within the `tutorials` subdirectory in the ALPS installation directory.

B. Python interface

Using `alpspython` together with the Python interface of the solver (the `cthyb` Python module) provides a very flexible framework for setting up any desired selfconsistency scheme. The selfconsistency is thereby implemented within a Python script which can be executed on a parallel machine using MPI. Two examples are given in the ALPS tutorials, i.e in the directories

```
06-hybridization/03-retarded-interaction
```

and

```
06-hybridization/04-spinfreezing
```

For how to run these tutorials, refer to Sec. III B.

VIII. SPECIFYING MEASUREMENTS

The following variables are measured by default:

- **Sign:** The Monte Carlo fermionic Sign.
- **order_histogram_total:** The histogram of expansion orders, containing all interaction vertices.
- **order_histogram_i:** with i from 0 to $N_{\text{orb}} - 1$: the expansion order in each orbital
- **density_i:** density in orbital i , with i from 0 to $N_{\text{orb}} - 1$.
- **g_i:** with i from 0 to $N_{\text{orb}} - 1$: Green's function in imaginary time, binned on `N_TAU` time slices between zero and β .

If the frequency measurement is enabled, the following variables are measured in addition:

- **gw_re_i:** with i from 0 to $N_{\text{orb}} - 1$: Green's function in frequency, real part.
- **gw_im_i:** with i from 0 to $N_{\text{orb}} - 1$: Green's function in frequency, imaginary part.

- `fw_re_i`: with i from 0 to $N_{\text{orb}} - 1$: F function for improved estimators in frequency, real part.
- `fw_im_i`: with i from 0 to $N_{\text{orb}} - 1$: F function for improved estimators in frequency, imaginary part.

If the legendre measurement is enabled, the following variables are measured in addition:

- `gl_i`: with i from 0 to $N_{\text{orb}} - 1$: Legendre coefficients of Green's function G .
- `fl_i`: with i from 0 to $N_{\text{orb}} - 1$: Legendre coefficients of F function for improved estimators.

These observables are written into a hdf5 file after the simulation exits. The standard hdf5 tools, `h5ls -r` and `h5dump` or `h5dump -d /simulation/results/Sign/mean/value` will give their values.

For all available measurements, we refer the reader to Sec. VI.

IX. ANALYSIS AND POST-PROCESSING

After the simulation exits, the simulation results have to be read in and evaluated. This is best done using one of the many hdf5 tools. The ALPS DMFT framework provides access from C++, but Python tools (`h5py` and similar) also work. The hybridization tutorials show how to use the ALPS Python hdf5 interface. Typically, the Green function, its error, and the densities need to be read in and post-processed.

A. Elementary checks: Expansion order and Sign problem

Each new run of the impurity solver should start with a check of the expansion order and the error on it. Plot the results of `order_histogram_i` including error bars. Does the statistics look good? is the expansion order reasonable? If there are degenerate orbitals, is their expansion order the same within error bars? For diagonal hybridizations, the Monte Carlo sign problem should be always one. As a rule of thumb: If your sign drops below about 0.75, you have to be careful. If your sign is below 0.1, your results are most likely too noisy to be useful.

B. Further checks: Errors of Green functions

In a next step, extract the Green function from the results and analyze them. Is the statistics good enough for your purposes? Do the error bars make sense? If you use `h5py`, you can extract the Green functions in τ from an output file `sim.h5` using

```
import h5py
h5_file = h5py.File("sim.h5", 'r')
g_mean = h5_file["/simulation/results/\
g_0/mean/value"].value
g_error = h5_file["/simulation/results/\
g_0/mean/error"].value
```

Once you trust your Green functions, Fourier transform to frequency and analyze the self energy (which will be used in the self-consistency equations). The self energies are very sensitive.

C. Fourier transforms and high frequency tails

To get smooth and correct behavior you may need to use high frequency Fourier transform tricks (see e.g. the review, Ref. 12). Further information can be found in the references cited in the review chapter X.I.

D. Running MaxEnt

The Maximum entropy method, or MaxEnt, is used to analytically continue imaginary frequency or imaginary time data to the real axis to produce spectral functions that are consistent with the imaginary time data within statistical errors.¹³ ALPS provides an implementation of a maximum entropy analytic continuation code. Note that MaxEnt is completely uncontrolled: no reliable error bars for the output exist. In addition, small differences in the input data will cause large differences in the output. Because of this, any conclusion drawn from continued data needs to be backed up by data on the imaginary axis, where error bars are available.

A correct error propagation that includes both the errors and the covariance matrices of the Green's function is absolutely essential for reliable continuations. Additionally, it helps to continue the self-energies instead of the Green's functions¹⁴.

A tutorial for the ALPS maxent code will be provided elsewhere. Fig. 1 shows a simple run file; `maxent --schema` lists all keys, and the number of data points follows from the length of `input.values`.

X. SCALING – ACCESSIBLE PROBLEMS

The algorithm scales cubically in matrix size (expansion order), and is therefore $O(n_o\beta^3)$ with a relatively small (but complicated) dependence of U . Typically problems with an expansion order of less than 400 per orbital are easy to do.

```

[parameters]
N_ALPHA = 60                # number of alphas between ALPHA_MIN and ALPHA_MAX
ALPHA_MIN = 0.05             # smallest alpha value to be evaluated
ALPHA_MAX = 5.0              # largest alpha value to be evaluated
NORM = 1.0                   # normalization of the function to be continued
OMEGA_MAX = 25.0             # frequency range, symmetric unless OMEGA_MIN is given
KERNEL = "fermionic"         # MaxEnt kernel
BETA = 60.0                  # inverse temperature
NFREQ = 2001                 # number of output frequencies
FREQUENCY_GRID = "Lorentzian"
DATASPACE = "frequency"      # "time" or "frequency"
MAX_IT = 2000                # iterations of the minimization procedure
DEFAULT_MODEL = "flat"
PARTICLE_HOLE_SYMMETRY = true

[input]
values = [-1.58151510066, -1.19058822793, ..., -0.0186270492461]
errors = [0.00526104728966, 0.00199036770537, ..., 3.42643209543e-07]

[output]
results = "spectrum.h5"

```

FIG. 1. A simple run file for the maxent code.

XI. WHAT IS NOT POSSIBLE WITH THE HYBRIDIZATION CODE

You cannot apply the code for calculations in the atomic limit, i.e. for vanishing hybridization.

The following problems are not feasible at the moment:

1. Problems with non-diagonal hybridization functions
2. Problems with general, non density-density interactions

It is planned to extend the code to the first two types of impurity problems. The second type of impurity problems may be considered at a later stage.

XII. LITERATURE AND FURTHER INFORMATION

- Dynamical mean field theory: please have a look at the original review¹⁰ and the review by Kotliar *et al.*¹¹. Introductory information can also be found in Antoine George's summer school lecture notes¹⁵.
- LDA+DMFT: Apart from Ref. 11, Refs. 16 and 17 are good places to start.

- Continuous-time methods: The review¹², along with summer school lecture notes¹⁸ and PhD theses¹⁹ and the original literature.^{2,20,21}

XIII. WHEN PROBLEMS APPEAR – TROUBLESHOOTING

This code is still in beta stage and problems will appear. To report a problem please contact the ALPS user mailing list or [Emanuel](#) via e-mail.

XIV. PUBLISHING – CITING THE ALPS HYBRIDIZATION EXPANSION CODE

Citations are important for us – they mean that we can justify investing time into the development of the hybridization expansion code and other open source projects. A paper should acknowledge the use of the ALPS libraries, the use of the hybridization code, and the original algorithm paper. A citation line could be:

Our simulations used an open source implementation³ of the hybridization expansion continuous-time quantum Monte Carlo algorithm² and the ALPS¹ libraries.

Or, alternatively:

Our simulations used the ALPS open source hybridization expansion code¹⁻³.

¹ B. Bauer *et al.*, *Journal of Statistical Mechanics: Theory and Experiment* **2011**, P05001 (2011).

² P. Werner, A. Comanac, L. de' Medici, *et al.*, *Phys. Rev. Lett.* **97**, 076405 (2006).

³ H. Hafermann, P. Werner, and E. Gull, *Computer Physics Communications* **184**, 1280 (2013).

⁴ E. Gull, P. Werner, S. Fuchs, B. Surer, T. Pruschke, and M. Troyer, *Computer Physics Communications* **182**, 1078

- (2011).
- ⁵ P. Werner and A. J. Millis, *Physical Review Letters* **99**, 146404 (2007).
 - ⁶ P. Werner and A. J. Millis, *Phys. Rev. Lett.* **104**, 146401 (2010).
 - ⁷ H. Hafermann, K. R. Patton, and P. Werner, *Phys. Rev. B* **85**, 205106 (2012).
 - ⁸ L. Boehnke, H. Hafermann, M. Ferrero, F. Lechermann, and O. Parcollet, *Phys. Rev. B* **84**, 075145 (2011).
 - ⁹ L. Huang and L. Du, “Kernel polynomial representation of imaginary-time green’s functions,” (2012), unpublished, [arXiv:1205.2791](https://arxiv.org/abs/1205.2791).
 - ¹⁰ A. Georges, G. Kotliar, W. Krauth, and M. J. Rozenberg, *Rev. Mod. Phys.* **68**, 13 (1996).
 - ¹¹ G. Kotliar, S. Y. Savrasov, K. Haule, *et al.*, *Rev. Mod. Phys.* **78**, 865 (2006).
 - ¹² E. Gull, A. J. Millis, A. I. Lichtenstein, A. N. Rubtsov, M. Troyer, and P. Werner, *Rev. Mod. Phys.* **83**, 349 (2011).
 - ¹³ M. Jarrell and J. E. Gubernatis, *Physics Reports* **269**, 133 (1996).
 - ¹⁴ X. Wang, E. Gull, L. de’ Medici, M. Capone, and A. J. Millis, *Phys. Rev. B* **80**, 045101 (2009).
 - ¹⁵ A. Georges, *LECTURES ON THE PHYSICS OF HIGHLY CORRELATED ELECTRON SYSTEMS VIII: Eighth Training Course in the Physics of Correlated Electron Systems and High-Tc Superconductors* **715**, 3 (2004).
 - ¹⁶ K. Held, I. A. Nekrasov, G. Keller, V. Eyert, N. Blüemer, A. K. McMahan, R. T. Scalettar, T. Pruschke, V. I. Anisimov, and D. Vollhardt, *Phys. Status Solidi* **243**, 2599 (2006).
 - ¹⁷ K. Held, *Advances in Physics* **56**, 829 (2007).
 - ¹⁸ P. Werner, Lecture notes for the International Summer School on Numerical Methods for Correlated Systems in Condensed Matter, Sherbrooke, Canada (2008).
 - ¹⁹ E. Gull, *Continuous-time quantum Monte Carlo algorithms for fermions*, *Ph.D. thesis*, ETH Zurich (2008).
 - ²⁰ A. N. Rubtsov, V. V. Savkin, and A. I. Lichtenstein, *Phys. Rev. B* **72**, 035122 (2005).
 - ²¹ A. N. Rubtsov and A. I. Lichtenstein, *JETP Letters* **80**, 61 (2004).